Lesson 06 Demo 03

Creating a React Application Using combineReducers

Objective: To develop a React application that uses combineReducers for efficient state management

Tools required: Node Terminal, React app, and Visual Studio Code

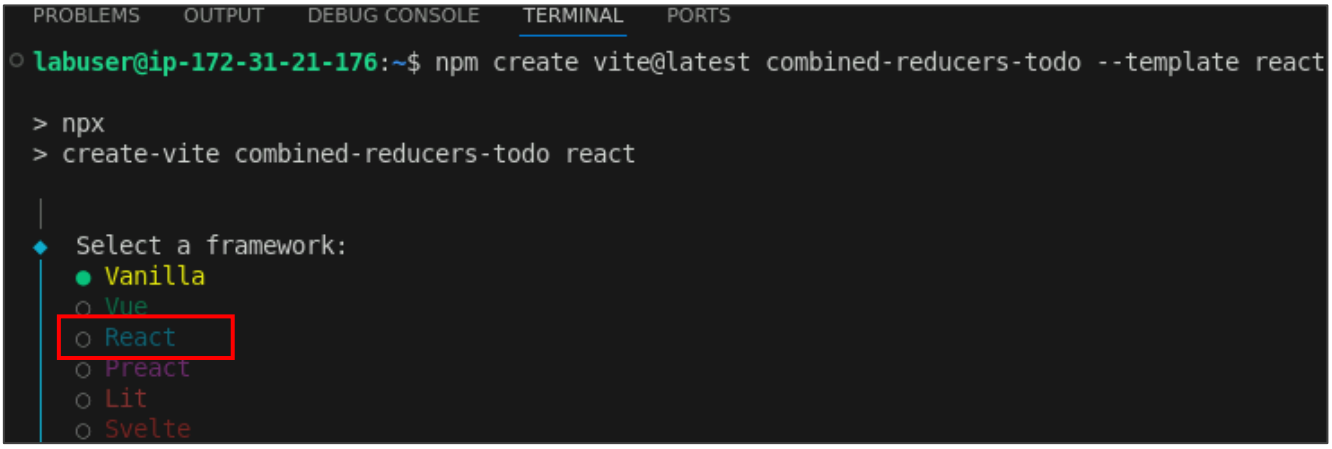
Prerequisites: Knowledge of creating a React app and understanding of the folder structure

Steps to be followed:

1. Create a new React app
2. Create a new file called reducers.js
3. Create a new file called AddTodo.js
4. Import the root Reducer from reducers.js into the index.js file
5. Run the application and view it in the browser

Step 1: Create a new React app

1.1 Open the terminal and run the command below to create a new project:
npm create vite@latest combined-reducers-todo --template react



1.2 Select **React** as the framework and **JavaScript** as the variant

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

◇ Select a framework:
  React

◆ Select a variant:
  ○ TypeScript
  ○ TypeScript + SWC
  ● JavaScript
  ○ JavaScript + SWC
  ○ React Router v7 ↗
```

1.3 Open the combined-reducers-todo directory by running the command below:
cd combined-reducers-todo

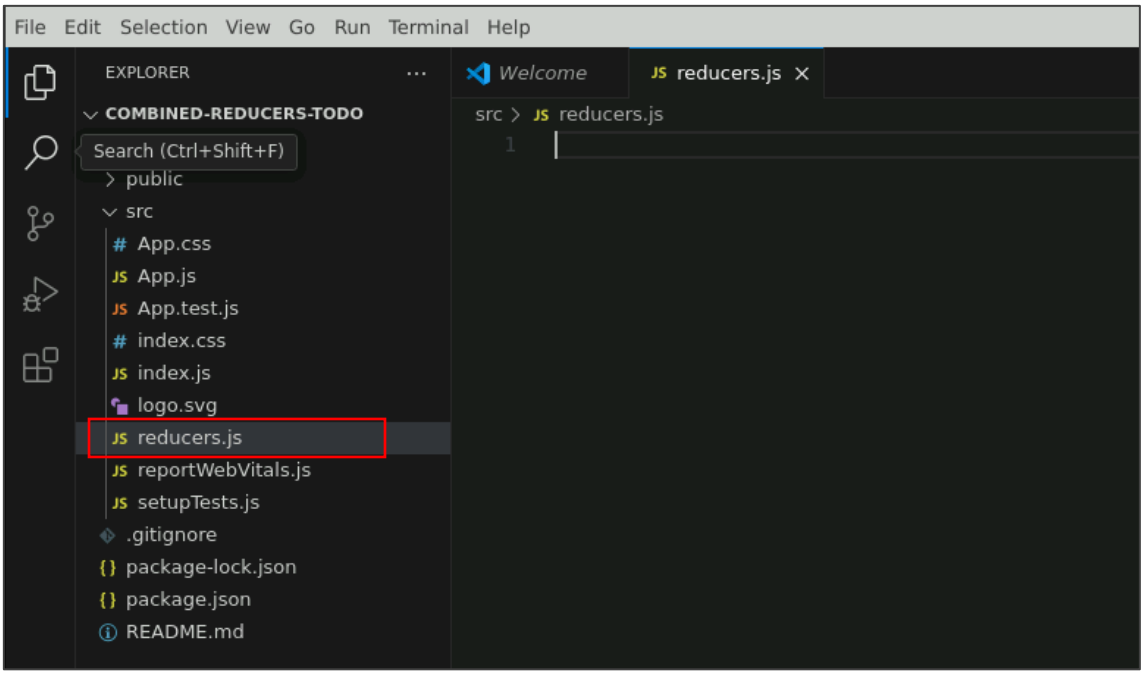
```
● labuser@ip-172-31-38-215:~$ cd combined-reducers-todo
○ labuser@ip-172-31-38-215:~/combined-reducers-todo$
```

1.4 Run the following command to install the necessary dependencies:
npm install redux react-redux

```
● labuser@ip-172-31-38-215:~/combined-reducers-todo$ npm install redux react-redux

  added 4 packages, and audited 1347 packages in 5s

  268 packages are looking for funding
    run `npm fund` for details

  12 vulnerabilities (6 moderate, 6 high)

  To address all issues (including breaking changes), run:
    npm audit fix --force
```

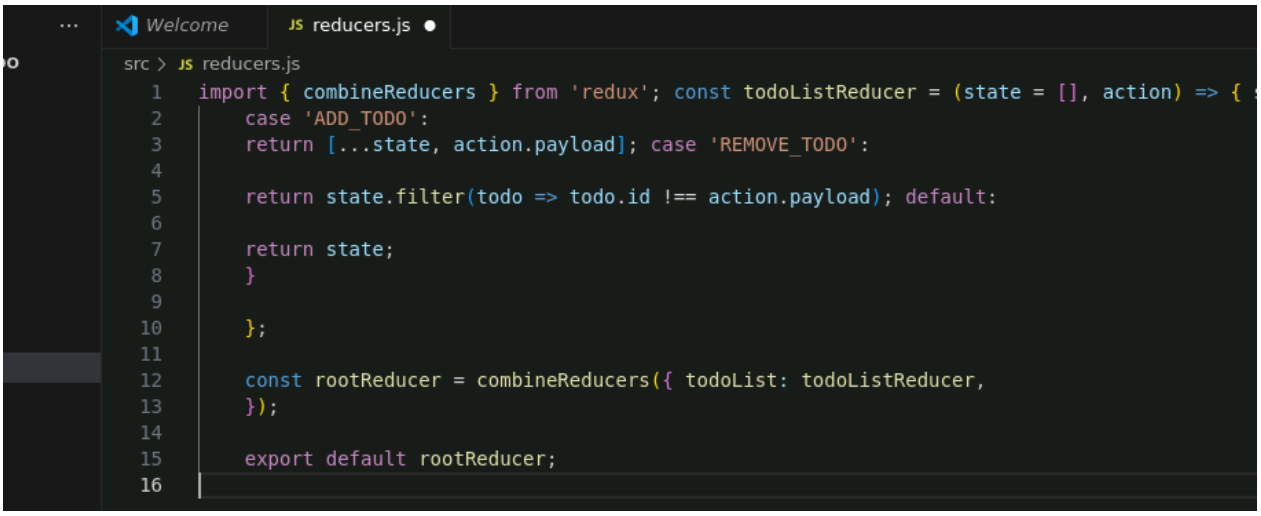
Step 2: Create a new file called reducers.js

2.1 Create a new file named **reducers.js** in the src directory

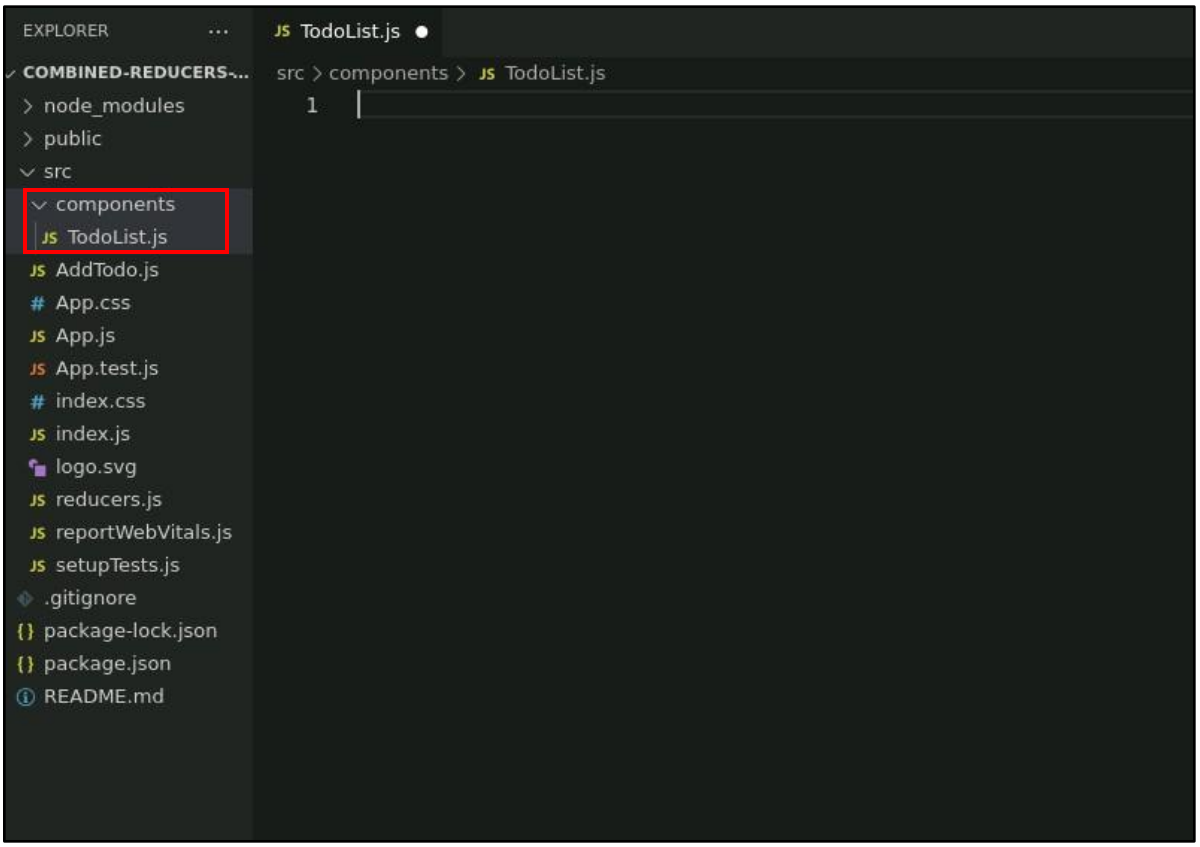


2.2 Enter the following code in the **reducer.js** file:

```
import { combineReducers } from 'redux';
const todoListReducer = (state = [], action) => {
switch (action.type) {
case 'ADD_TODO':
return [...state, action.payload];
case 'REMOVE_TODO':

return state.filter(todo => todo.id !== action.payload);
default:

return state;
}

};

const rootReducer = combineReducers({
todoList: todoListReducer,
});

export default rootReducer;
```

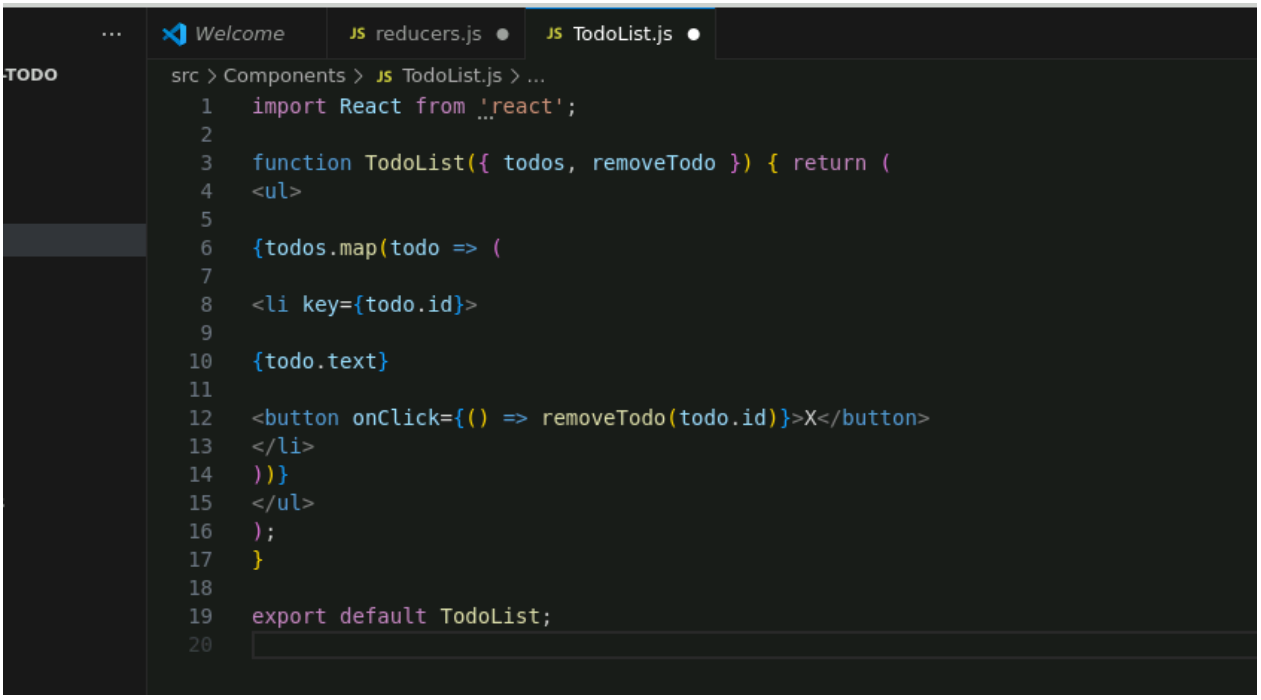


2.3 In the **src/components** directory, create a file named **TodoList.js**



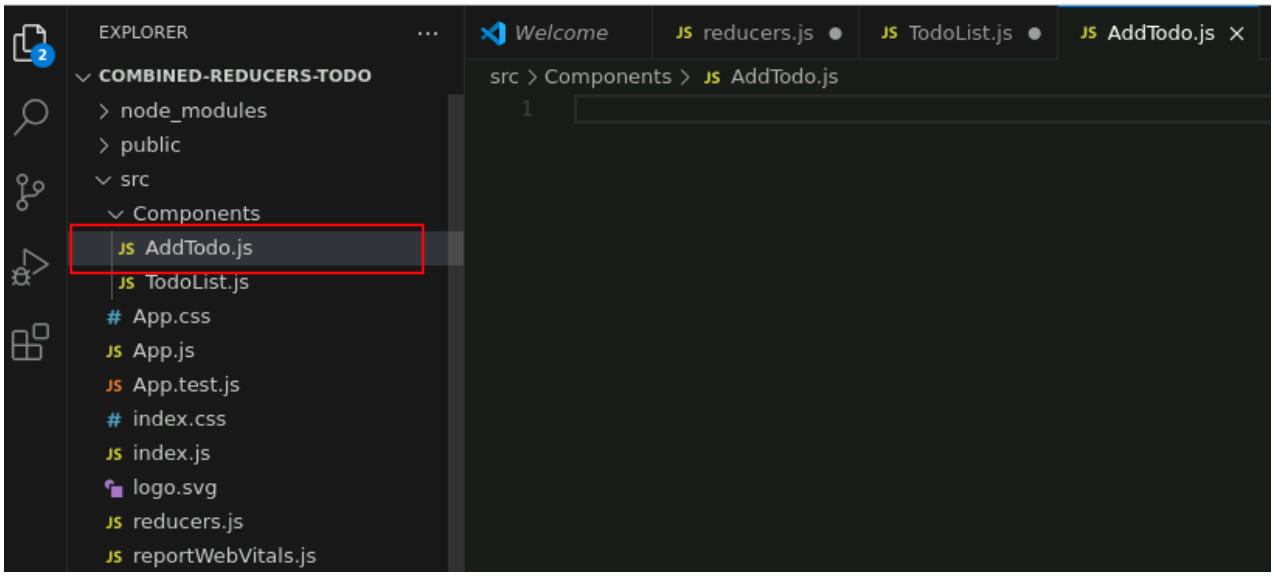
2.4 Enter the following code in the **TodoList.js** file:

```
import React from 'react';

function TodoList({ todos, removeTodo }) {
return (
<ul>

{todos.map(todo => (

<li key={todo.id}>

{todo.text}

<button onClick={() => removeTodo(todo.id)}>X</button>
</li>
))}
</ul>
);
}

export default TodoList;
```



Step 3: Create a new file called AddTodo.js

3.1 In the **src/components** directory, create a new file named **AddTodo.js**



3.2 Enter the following code in the **AddTodo.js** file:

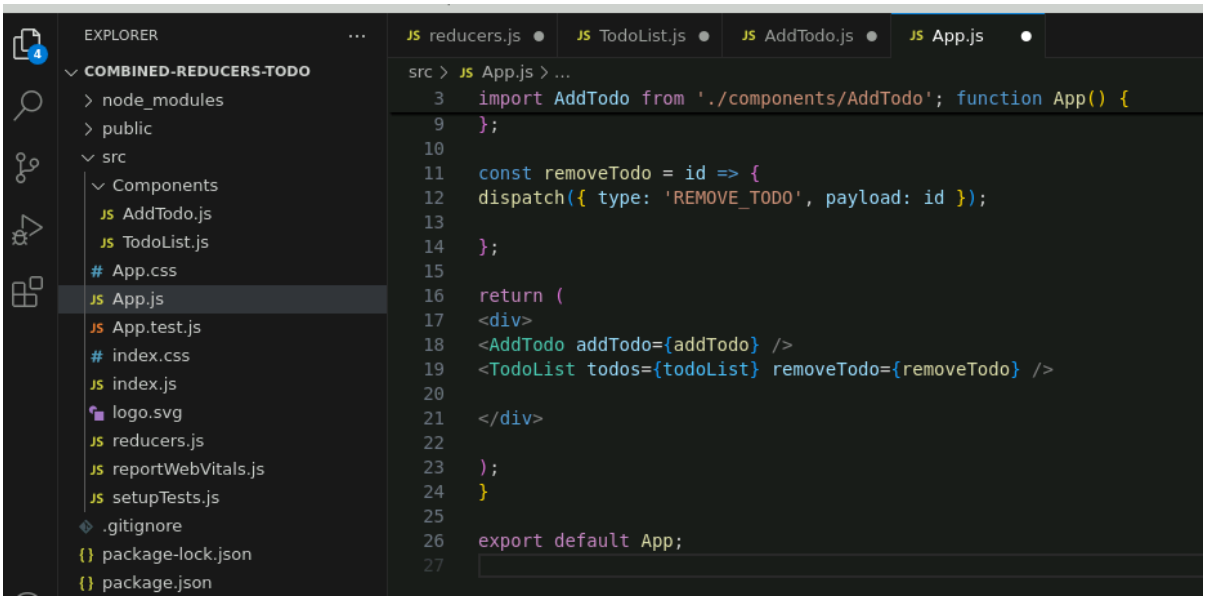
```
import React, { useState } from 'react';
function AddTodo({ addTodo }) {
const [text, setText] = useState('');
const handleSubmit = e => {
e.preventDefault();
addTodo({
id: Date.now(),
text,
});
setText('');
};
return (
<form onSubmit={handleSubmit}>
<input type="text" value={text} onChange={e =>
setText(e.target.value)} />
<button type="submit">Add Todo</button>
</form>
);
}
export default AddTodo;
```

```
Welcome   reducers.js   TodoList.js   AddTodo.js
src > Components > AddTodo.js > ...
1  import React, { useState } from 'react'; function AddTodo({ addTodo }) {
2      const [text, setText] = useState('');
3
4      const handleSubmit = e => { e.preventDefault(); addTodo({
5      id: Date.now(), text,
6      });
7      setText('');
8      };
9      return (
10     <form onSubmit={handleSubmit}>
11     <input type="text" value={text} onChange={e => setText(e.target.value)} />
12     <button type="submit">Add Todo</button>
13     </form>
14     );
15     }
16     export default AddTodo;
17
```

3.3 In the src directory, open the existing file named **App.js** and enter the following code:

```
import React from 'react';
import { useSelector, useDispatch } from 'react-redux';
import TodoList from './components/TodoList';
import AddTodo from './components/AddTodo';
function App() {
const todoList = useSelector(state => state.todoList);
const dispatch = useDispatch();

const addTodo = todo => {
dispatch({ type: 'ADD_TODO', payload: todo });
};

const removeTodo = id => {
dispatch({ type: 'REMOVE_TODO', payload: id });
```

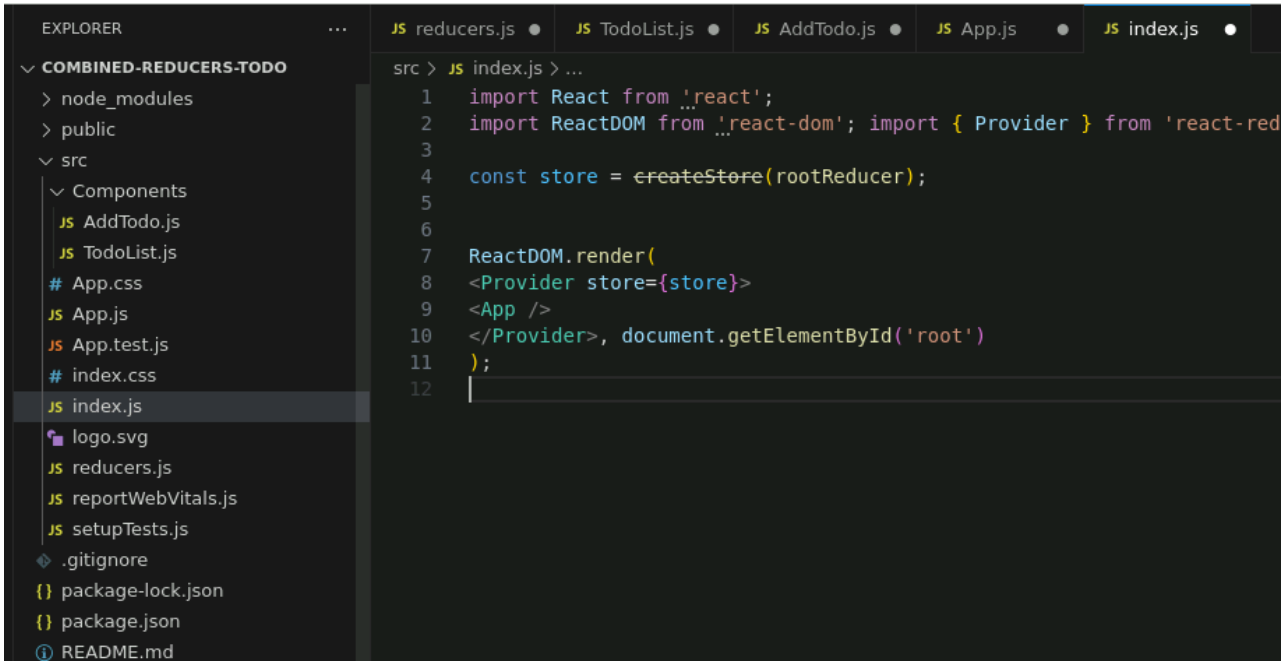
```
};

return (
<div>
<AddTodo addTodo={addTodo} />
<TodoList todos={todoList} removeTodo={removeTodo} />

</div>

);
}

export default App;
```



Step 4: Import the rootReducer from reducers.js into the index.js file

4.1 In the src directory, open the existing **index.js** file and enter the following code:

```
import React from 'react';
import ReactDOM from 'react-dom'; import {
Provider } from 'react-redux'; import {
createStore } from 'redux'; import
rootReducer from './reducers'; import App
from './App';

const store = createStore(rootReducer);


ReactDOM.render(
<Provider store={store}>
<App />
</Provider>,
document.getElementById('root')
);
```



Step 5: Run the application and view it in the browser

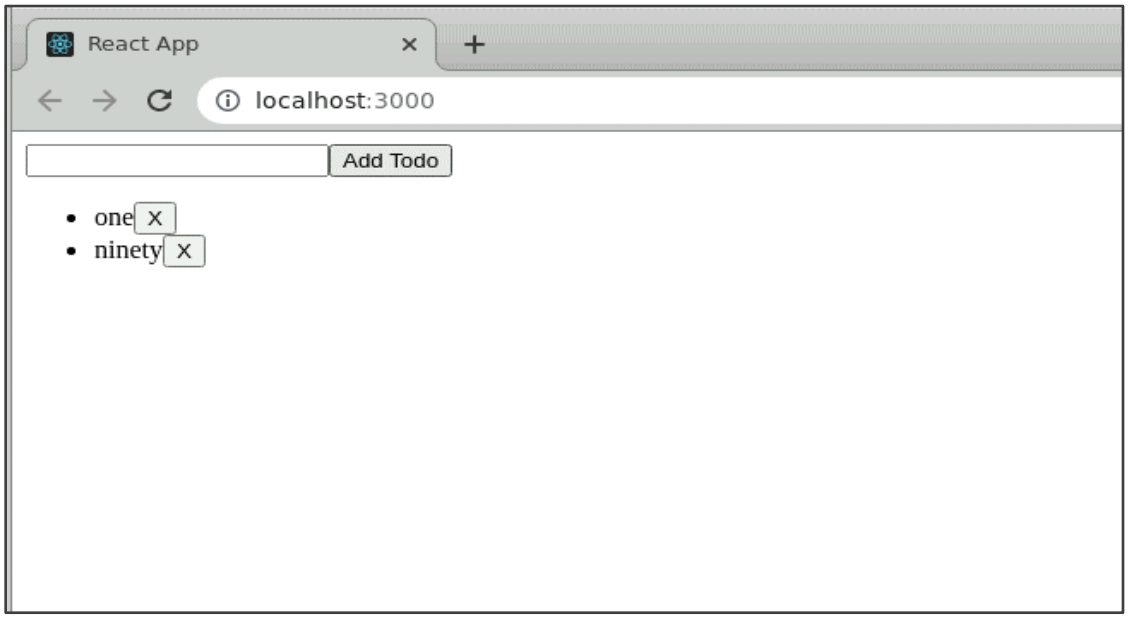
5.1 Run the following command to start the application:

npm start

```
labuser@ip-172-31-38-215:~/combined-reducers-todo$ npm start

 > combined-reducers-todo@0.1.0 start
 > react-scripts start
```

5.2 Open your browser and navigate to the following link:

http://localhost:3000



The code above demonstrates a systematic process initiating a React app, configuring reducers in reducers.js for state management.

By following these steps, you have successfully created a React application using combineReducers for state management.